

RAILWAY TRAFFIC ANALYSIS

Sophie Pull
Dixant Pant
Anthony Oparaocha
George Hunt
Minh Le
Mohsin Zeeshan Syed



9TH MAY 2025
FINAL SEP VIVA

Context & Vision

SYNOPTIX

- Modelling of railway data in order to gain a deeper understanding of the variables that affect the probability of railway incidents.
 - Incidents include: derailments, collisions, obstructions etc.
 - Variables include: weather conditions, temperature, track type, train type etc.
- Allows our **client - Synoptix** (an engineering consultancy) - to develop technologies to mitigate these variables and justify the value of their technologies to their customers.

Synoptix

As **Synoptix**, we want to identify variables that increase railway incident probability, so that we can **develop targeted, data-driven technologies**, as well as **demonstrate and justify their value and effectiveness to stakeholders**.



Railway Commuters

As a **railway commuter**, I want companies to adopt Synoptix technology to prevent incidents, so that I can feel **safer and more confident** when using the railway system.

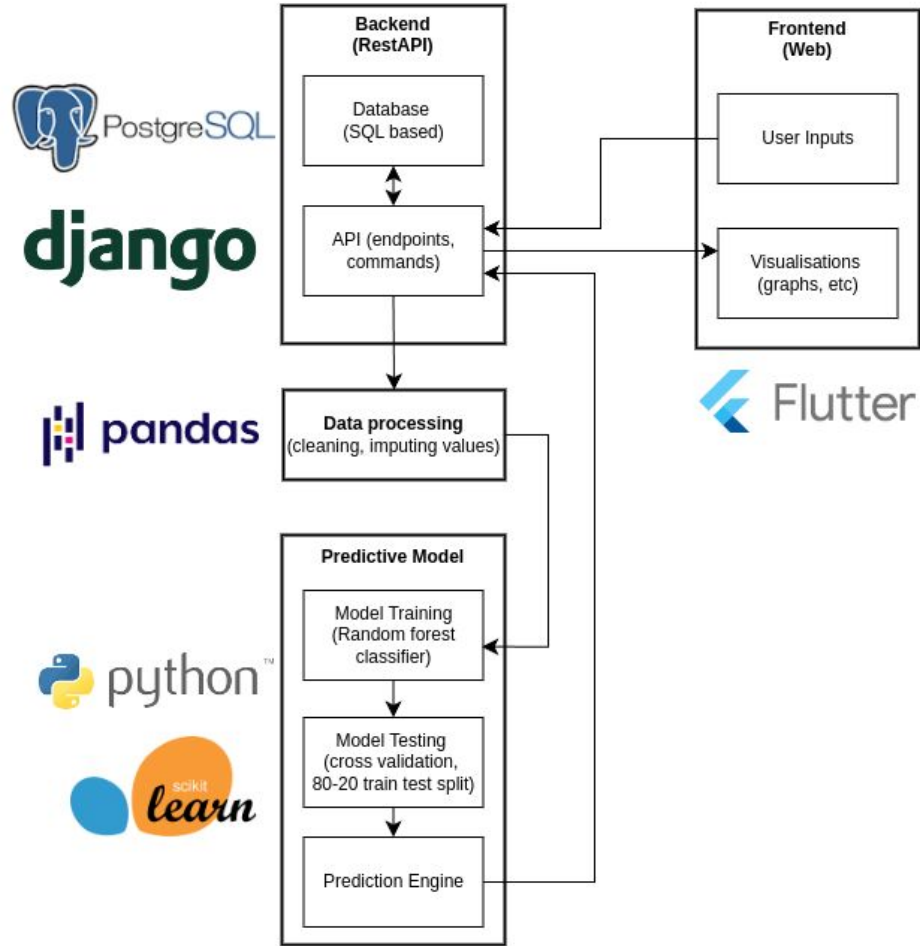


Railway Companies

As a **railway company**, we want to **reduce incidents** and associated **costs**, such as rail repairs, train repairs, and insurance expenses, so that we can **prioritise safety**, **prevent casualties**, **build public trust**, and maintain a reliable, **positive brand image**.



Tech Stack + Architecture



CI/CD



- Hosted on an AWS EC2 instance
 - Contains dockerized containers of Flutter and Django code and an upstream container for PostgreSQL
 - Updates via a CD pipeline that run on push to main to update the Docker containers in the EC2 instance
- Used Cloudflare to provide SSL/TLS certification to ensure secure connections
- Comprehensive tests in continuous integration
 - Includes linting for both frontend and backend

Coverage report: 84%

Files Functions Classes

coverage.py v7.6.12, created at 2025-02-25 10:07 +0000

File	statements	missing	excluded	coverage
manage.py	11	2	0	82%
rtapi/__init__.py	1	0	0	100%
rtapi/admin.py	12	0	0	100%
rtapi/apps.py	4	0	0	100%
rtapi/management/__init__.py	0	0	0	100%
rtapi/management/commands/__init__.py	0	0	0	100%
rtapi/management/commands/clean_data.py	39	5	0	87%
rtapi/migrations/__init__.py	0	0	0	100%
rtapi/migrations/0001_initial.py	6	0	0	100%
rtapi/migrations/0002_userinput_mainaccidentcauses_description.py	4	0	0	100%
rtapi/migrations/0003_remove_mainaccidentcauses_description.py	4	0	0	100%
rtapi/migrations/0004_mainaccidentcauses_description_and_more.py	4	0	0	100%
rtapi/migrations/0005_userinput_gross_tonnage.py	4	0	0	100%
rtapi/migrations/0006_rename_track_direction_userinput_train_direction.py	4	0	0	100%
rtapi/models.py	134	11	0	92%
rtapi/multinomial_logit.py	32	25	0	22%
rtapi/random_forest_model.py	37	13	0	65%
rtapi/serializers.py	46	0	0	100%
rtapi/tests/__init__.py	0	0	0	100%
rtapi/tests/test_scripts.py	120	2	0	98%
rtapi/tests/tests.py	104	0	0	100%
rtapi/urls.py	5	0	0	100%
rtapi/views.py	93	43	0	54%
rtapi/__init__.py	1	0	0	100%
rtapi/asgi.py	4	4	0	0%
rtapi/settings.py	29	1	0	97%
rtapi/urls.py	6	0	0	100%
rtapi/wsgi.py	4	4	0	0%
Total	708	110	0	84%

coverage.py v7.6.12, created at 2025-02-25 10:07 +0000

Backend
Code Coverage:
84%

Frontend
Code Coverage:
73.7%

LCOV - code coverage report

Current view: top level

Test: lcov.info

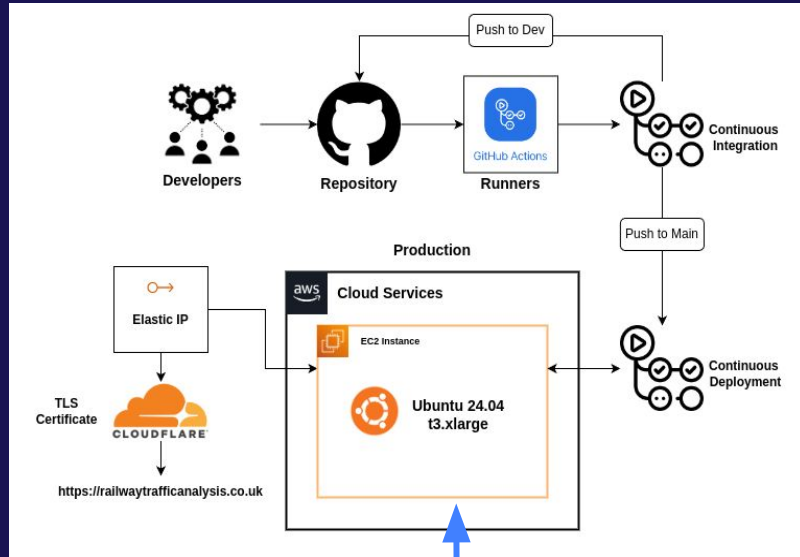
Test Date: 2025-04-03 02:12:59

	Coverage	Total	Hit
Lines:	73.7 %	1818	1339
Functions:	-	0	0

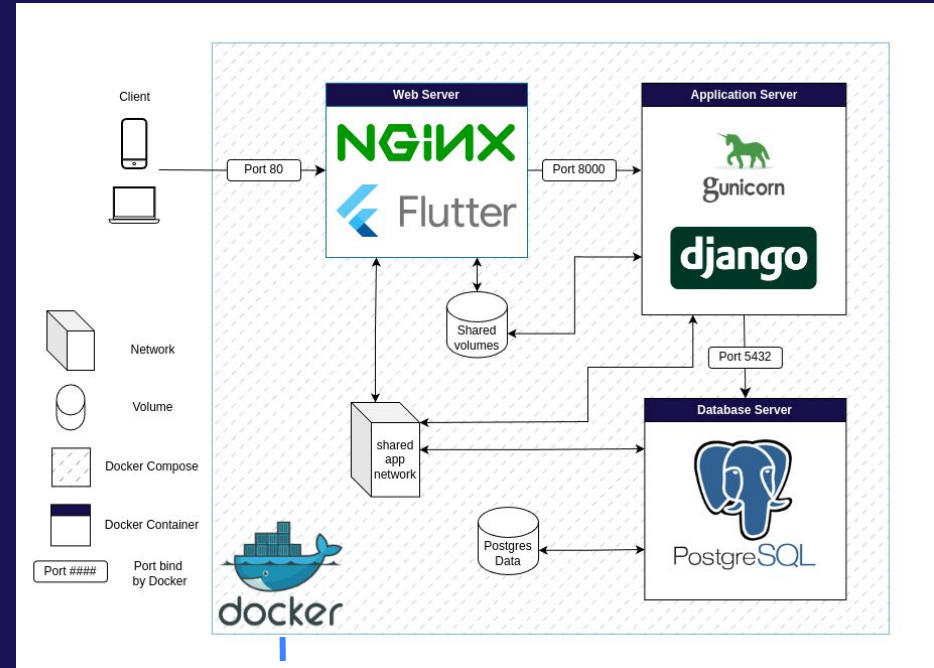
Directory	Line Coverage ↕		
	Rate	Total	Hit
lib/		835	530
lib/components/		448	376
lib/models/		6	3
lib/pages/		309	220
lib/providers/		201	193
lib/themes/		19	17

Note: 'Function Coverage' columns elided as function owner is not identified.

CI/CD Workflow and AWS EC2 Deployment



Dockerized Web Application Architecture



Dockerized Web Application Running Inside EC2
(via Docker Compose)

Interface Designs (Old)

Railway Traffic Analysis

Please enter risk factor parameters into the fields below. We will then determine the associated risks based on these factors.

Amount of rain (mm):

Temperature (degrees C):

Proximity of crossing (km):

Train speed (mph):

Wind speed (mph):

SYNOPTIX

SYNOPTIX

Railway Traffic Analysis

Train External Data

Type of weather

Clear

Precipitation

mm

Temperature

°C

Location

Time

Time

Train Internal Data

Type of Train

Train Speed 0-250 Mph

Accident in the last 5 years ☐

Angle of the Train

Number of Signals

- Lacks validation or restrictions on user-entered data
- No dropdown menus to guide user input
- Limited visual appeal or user-friendly design

Interface Designs (Old)

SYNOPTIX

Railway Traffic Analysis

Train External Data

Weather Condition

Cloudy

Track Type

Visibility

Track Direction

Signalization

Train Internal Data

Equipment

Method of Operation

Submit

MVP Interface

SYNOPTIX

Railway Traffic Analysis

Train External Data

Weather Condition

Track Type

Visibility

Track Direction

Signalization

☒ °F

☐ °C

Temperature

0/3

Train Internal Data

Equipment

Method of Operation

☒ mph

☐ km/h

Train Speed

0/3

Equipment attended

Submit

Testing Day

Testing Day (31st Jan) feedback

What went well:

- Intuitive user interface
- Users like the dropdown menus and input validation (e.g no one was able to put text into numerical fields!)

Issues which we have since addressed:

- User input bugs (no temperature value validation, doesn't allow decimals, doesn't allow user to change fields back to being empty)
- Students unaware of train definitions
- Unit buttons (change between F and C) in unintuitive position
- No user instructions

Was anything difficult to find?

The size of the submit button is too small

Submit button too small, unit buttons not clear enough, need guidance on temperature & speed value

The Submit button

Submit button

I had to scroll down to the submit button

Was there anything unclear?

The units for temperature and speed were not in intuitive positions

Units should be closer to input fields

Units for Km and mph

Km, mph, K and degrees C should be closer

Feedback Implementation

2024-RailwayTrafficAnalysis #185

[BUG]: Temperature input deletes input when anything but a number is inputted into it

bug

testing day

PO

#196

[Sprint2] Beta Release

2024-RailwayTrafficAnalysis #187

[BUG]: Allow the users to change fields back to being empty

bug

testing day

PO

#192

[Sprint2] Beta Release

2024-RailwayTrafficAnalysis #188

[BUG]: Make the temperature field more realistic

bug

testing day

PO

#196

[Sprint2] Beta Release

2024-RailwayTrafficAnalysis #226

[FEATURE]: Create info for user about input variables

M

enhancement

frontend

testing day

P1

#242

[Sprint2] Beta Release

2024-RailwayTrafficAnalysis #284 ...

[FEATURE]: Refactor output page colour scheme

XS

enhancement

frontend

testing day

P2

[Sprint3] Post-Testathon

2024-RailwayTrafficAnalysis #196

Temperature and speed sensible inputs, submit not possible when no inputs, temp and speed inputs now don't delete,

bug

frontend

testing day

[Sprint2] Beta Release

2024-RailwayTrafficAnalysis #194

[BUG]: Make the speed field more realistic

S

bug

frontend

testing day

PO

#196

[Sprint2] Beta Release

2024-RailwayTrafficAnalysis #186

[BUG]: Make sure that you can't press submit with no entries made into the form

S

bug

testing day

PO

#196

[Sprint2] Beta Release

2024-RailwayTrafficAnalysis #182

[FEATURE]: Create a mock-up for an interactive UI

M

frontend

testing day

P1

#243

[Sprint2] Beta Release

2024-RailwayTrafficAnalysis #223

Attend Testathon and gather feedback on improvements

S

documentation

testing day

P1

[Sprint2] Beta Release

2024-RailwayTrafficAnalysis #290

[BUG]: Make track density field not delete inputs when unpermitted symbol typed

XS

bug

frontend

testing day

P1

#294

[Sprint3] Post-Testathon

Testathon (5th March) feedback



SYNOPTIX
Railway Traffic Analysis

Train External Data
Weather Condition
Track Type
Visibility
Track Direction
Signalization
☒ °F ☐ °C
Temperature
0/3

Train Internal Data
Equipment
Method of Operation
☒ mph ☐ km/h
Train Speed
0/3
Equipment attended

Submit

At Testing Day

SYNOPTIX
Railway Traffic Analysis

Train External Data
Weather condition
Track type
Main
Track density
Annual track-density; gross tonnage in millio...
0/3
Visibility
Train direction
Track Class

Train Internal Data
Equipment
Train speed
mph km/h
0/3
Time of Accident
10:30 PM
Equipment attended
No

SUBMIT

At Testathon (Beta)

PROJECT DEMO

railwaytrafficanalysis.co.uk



Demo login credentials

Username: test

Password: welovesep!

How does the model work?



- Using a *Random Forest classifier*
 - "Given an accident occurs, what type is it?" given the input parameters
 - Why? Can be used in predicting *multiple* categories (in our case, accident types)
- Trained on dataset from the Federal Railway Administration
 - Wrote scripts to download, clean and preprocess the data from their website.
 - Using **80-20 train-test** split
- Initially *removed* rows with missing values - 220k+ rows of data to 9k rows.
 - Achieved 41% accuracy on test dataset
 - So to improve accuracy, we decided to **impute** values
- Achieved 68.2% accuracy using imputed dataset

```
"Accident Type_Broken train collision": 0,  
"Accident Type_Derailment": 0.29,  
"Accident Type_Explosion-detonation": 0,  
"Accident Type_Fire/violent rupture": 0.01,  
"Accident Type_Head on collision": 0.01,  
"Accident Type_Hwy-rail crossing": 0.15,  
"Accident Type_Obstruction": 0.24,  
"Accident Type_Other (describe in narrative)": 0.1,  
"Accident Type_Other impacts": 0.14,  
"Accident Type_RR grade crossing": 0,  
"Accident Type_Raking collision": 0.01,  
"Accident Type_Rear end collision": 0.02,  
"Accident Type_Side collision": 0.03
```

Client Feedback

SYNOPSIS

- Our client was very happy with our project, especially relating to our direct implementation of the **system requirements** for the project which we outlined with them at the beginning of the year:
 - *Provide a visualisation of a probabilistic distribution of rail incidents given certain input variables.*
 - *Be able to experiment with different input variables, to see which ones affect the probability of incidents the most (interactive UI)*
- Client liked how the pages were clear and intuitive.
- Values the additional features we implemented such as stats and crash insights pages.

Issues and Solutions - Project Management

Issues

- 1 No designated/assigned Project Managers - lack of direction >
- 2 Weekly team meeting to decide on tasks (self-assigned) >
- 3 Messaging on WhatsApp - messages getting lost, team members missing meetings >

Solutions

- 1 Introduced **alternating Project Managers** - 2 x per sprint, who assigns Kanban issues
-

Kanban Board

○ **Todo** 8 Estimate: 0

This item hasn't been started

2024-RailwayTrafficAnalysis #180

[DOCUMENTATION]: Reasoning for using ML model instead of stats model

S documentation testing day P1
Final Release

2024-RailwayTrafficAnalysis #283

[FEATURE]: Research into / refactor output page

enhancement frontend P1 #292
[Sprint3] Post-Testathon

2024-RailwayTrafficAnalysis #326

[FEATURE]: Display the real predictions in Bar Chart

M enhancement frontend P1
[Sprint4]

2024-RailwayTrafficAnalysis #329

[FEATURE]: Impute values for missing user input

M backend P1 [Sprint4]

○ **In progress** 10 / 6 Estimate: 0

This is actively being worked on

2024-RailwayTrafficAnalysis #291

[TASK]: Get approximate distribution of each condition for trains without accident from accidents that are probably independent of conditions

M pred-model P0
[Sprint3] Post-Testathon

2024-RailwayTrafficAnalysis #286

[TASK]: Improve output page modularity

M enhancement frontend P0 #292
[Sprint3] Post-Testathon

2024-RailwayTrafficAnalysis #301

[BUG]: Make it so you cannot press submit when not all input fields are filled in

S bug frontend P1 #309
[Sprint3] Post-Testathon

2024-RailwayTrafficAnalysis #314

[FEATURE]: Rewrite user input tests

M backend enhancement P1

○ **Testing / Reviewing** 14 / 6

Estimate: 0
This item is in review / testing

2024-RailwayTrafficAnalysis #298

[FEATURE]: Add linting for Flutter

M frontend P1 #321
[Sprint3] Post-Testathon

2024-RailwayTrafficAnalysis #297

[FEATURE]: Add linting for Django

M backend P1 #321
[Sprint3] Post-Testathon

2024-RailwayTrafficAnalysis #318

[Other issue request]: Fix flake8 linting issues for Django files

XL backend P1 #321
[Sprint3] Post-Testathon

2024-RailwayTrafficAnalysis #279

[FEATURE]: Adding additional tests for the output page

M frontend testing P1 #288

○ **Done** 260 Estimate: 0

This has been completed

2024-RailwayTrafficAnalysis #206

[FEATURE]: send predictions from pred-model to frontend

XL backend pred-model P0 #247
[Sprint2] Beta Release

2024-RailwayTrafficAnalysis #214

[BUG]: CI pipeline not working with Neon

L bug P0 #191
[Sprint2] Beta Release

2024-RailwayTrafficAnalysis #60

[FEATURE]: Create Initial Predictive Model

XL backend enhancement P0 #82
Minimum Viable Product MVP

2024-RailwayTrafficAnalysis #202

Implement CD

XL admin enhancement P0
[Sprint2] Beta Release

Issues and Solutions - Project Management

Issues

- 1 No designated/assigned Project Managers - lack of direction >
- 2 Weekly team meeting to decide on tasks (self-assigned) >
- 3 Messaging on WhatsApp - messages getting lost, team members missing meetings >

Solutions

- 1 Introduced **alternating Project Managers** - 2 x per sprint, who assigns Kanban issues
- 2 Introduced **sprints**: weekly **scrum** meetings and pair programming, biweekly **sprint reviews**

Example - Sprints and Sprint Review/Retro

Sprint Review & Retrospective 3 - [Post-Testathon]

Sprint Details

- Sprint Number: 3
- Start Date: [19-02-2025]
- End Date: [07-03-2025]
- Project Manager(s): George and Dixant

Sprint Review

🎯 Goals & Objectives

- Implement changes from Testathon feedback
- Fully integrate user input into predictive model

✅ Completed Work:

- Fix CD pipeline
- Integrate user input into the predictive model, instead of dummy data
- Create tests for information page

🚧 In Progress:

- Create tests for output page - we are refactoring the output page first (following feedback from mentor)
- Finish login system -> discuss use of JavaScript (in person meeting)
- Refactor frontend to use ThemeData

🚫 Blocked/Not Completed:

- Implement login system -> need to discuss use of JavaScript!

Sprint Retrospective

💡 What went well?

- Workload becoming progressively more equal
- We are continuing to do proper PRs -> pulling the code locally, running the codebase, testing the feature (i.e actually doing `flutter run`)

Minimum Viable Product MVP

Closed on Jan 20 ⌚ Last updated 3 months ago

100% complete 0 open 70 closed

[Edit](#) [Reopen](#) [Delete](#)

Latest release date for the Minimum Viable Product Release

[Sprint1] Predictive Model Launch

Closed on Feb 25 ⌚ Last updated about 1 month ago

100% complete 0 open 65 closed

[Edit](#) [Reopen](#) [Delete](#)

This sprint focuses on establishing the core infrastructure and fun...(more)

[Sprint2] Beta Release

Closed on Feb 25 ⌚ Last updated about 1 month ago

100% complete 0 open 75 closed

[Edit](#) [Reopen](#) [Delete](#)

Beta Release Date

Project Managers: Sophie and Dixant

[Sprint3] Post-Testathon

Closed on Mar 31 ⌚ Last updated about 1 month ago

100% complete 0 open 43 closed

[Edit](#) [Reopen](#) [Delete](#)

This sprint focuses on: Implementing UI changes from testing day a...(more)

[Sprint4]

Closed on Mar 31 ⌚ Last updated about 1 month ago

100% complete 0 open 36 closed

[Edit](#) [Reopen](#) [Delete](#)

Features to add: Backend: Impute values for predictive model - dat...(more)

[Sprint5] - Final Release

Closed 1 minute ago ⌚ Last updated less than a minute ago

100% complete 0 open 78 closed

[Edit](#) [Reopen](#) [Delete](#)

Final release date for all SEP projects Final tasks to complete: M...(more)

Issues and Solutions - Project Management

Issues

- 1 No designated/assigned Project Managers - lack of direction >
- 2 Weekly team meeting to decide on tasks (self-assigned) >
- 3 Messaging on WhatsApp - messages getting lost, team members missing meetings >

Solutions

- 1 Introduced **alternating Project Managers** - 2 x per sprint, who assigns Kanban issues
- 2 Introduced **sprints**: weekly **scrum** meetings and pair programming, biweekly **sprint reviews**
- 3 Moved to **Teams** with specific **channels**. Used **PRs** to discuss code. Used more consistent documentation style (**NumPy** and **reST docstrings**)

Example - Communicating with Teams



grp-SEP - Railway Traffic Analysis

▼ Main Channels

General

Backend

Frontend

Other discussions

Predictive Model

AO Anthony Oparaocha 1/22 8:18 PM

Review PR

Dixant Pant, **Sophie Pull** whenever free could you guys review my PR and tell me what you think about it. Thanks 🙏



SP Sophie Pull 1/23 5:36 PM

Will be free tmrw morning, if it's still open I'll take a look then;



SP Sophie Pull 1/25 9:44 AM

Anthony Oparaocha just checking is it this PR <https://github.com/spe-uob/2024-RailwayTrafficAnalysis/pull/131> ?

AO Anthony Oparaocha 1/25 3:21 PM
Yes it is 😊

ML Minh Le 1/24 3:06 PM Edited



No data for accidents that didn't happen

I don't think we ever formally discussed this or had a clear cut solution to this. So basically the problem is we don't have the records of when no accidents happened, so "no accidents" wouldn't be an output among the other output predictions like "derailment" or "collision", so our prediction is going to be something more like "given that there was an accident and there was this much visibility, the track type was this, etc., for each type of accident, what's the probability that the accident was of that type?" (which requires the sum of probabilities over all accident types to be 1) while we want something more like "given this much visibility, this track type, etc., for each type of accident, what's the probability that it's going to happen?". These are two different objectives and we can't substitute one for the other, or else it's gonna result the predictions implying things like "if you increase the trainspeed to 480km/h then the probability of a "collision" accident will reduce by 80%" when in actuality that is only true because the accident is always going to be "derailment" with a trainspeed that high, for example.

[see more](#)



Open 8 replies from Minh Le, you and Dixant Pant

SP Sophie Pull 1/25 9:46 AM

Oh yeah that "2 step" approach sounds interesting. I'm pretty sure I saw a research paper on random forests for predicting railway incidents done before, so it's definitely possible. I'm thinking we should take a look at their method and sorta work off that



Example - PR Discussions

dixantp commented on Apr 1 • edited • Author • ...

Tried it again and got this

```
python manage.py load_csv
SystemCheckError: System check identified some issues:

ERRORS:
rts_api.incident.equipment_attended: (fields.E128) CharField must define a 'max_length'
rts_api.incident.equipment_type: (fields.E128) CharField must define a 'max_length'
rts_api.incident.signalization: (fields.E128) CharField must define a 'max_length'
rts_api.incident.time: (fields.E128) CharField must define a 'max_length' attribute
```

Says the same thing when I thought it's a migrations issues and tried to either migrate or makemigrations, is it just the incidents model char fields not having a max length? Not sure what to do to get rid of this.

You should just be trying `python manage.py migrate` btw, makemigrations are only when you make changes, migrate is for when you want to apply changes that have been made in new migrations I'm pretty sure.

Not sure why this error is happening though, do you have a postgresql backend set up properly, but this surely is fixed now that [@sophiepull](#) added max lengths back to the models. Just a bit confused why it's giving you that error but not giving it to me and [@sophiepull](#)

dixantp requested a review from **minihle-bristol** 2 months ago

dixantp linked an issue on [Apr 1](#) that may be closed by this pull request

[FEATURE] Create a script to load the Form 54 csv into the database #397

sophiepull commented on Apr 1 • edited • ...

So this would also solve the incidentkey issue as well as we are not using the cleaned data.

Not entirely. The command that Minih has written to download the data from the FRA API is indeed the unclear data, its just that it only downloads the fields we need for training the fields like time, accident type etc...) and this doesn't include the incident key.

sophiepull commented on Apr 1 • edited • ...

You should just be trying `python manage.py migrate` btw, makemigrations are only when you make changes, migrate is for when you want to apply changes that have been made in new migrations I'm pretty sure.

Yep this is correct. When you change the file `models.py`, you then have to run `makemigrations`, which will make you a new `migrations` file which you then must commit to the repo. Then running `migrate` will actually run those migration files that have been created.

But nothing bad will happen if you just run `makemigrations` without changing the model, its just redundant.

Not sure why this error is happening though, do you have a postgresql backend set up properly, but this surely is fixed now that [@sophiepull](#) added max lengths back to the models. Just a bit confused why it's giving you that error but not giving it to me and [@sophiepull](#)

I think it might have something to do with this:

The maximum length (in characters) of the field. The max_length is enforced at the database level and in Django's validation using `MaxLengthValidator`. It's required for all database backends included with Django except PostgreSQL which supports unlimited VARCHAR columns.

From: [Django docs](#)

Must be something to do with not having PostgreSQL on Minih's side? Either way, cant hurt to have some extra rules about max length.

GeorgeHuntBris reviewed on Mar 18 View reviewed changes

GeorgeHuntBris left a comment

This PR will be v useful well done. I think filling numerical columns with mean values and categorical with the most frequent category is a good strategy for imputing values. And as you were saying about trying to get the time to reflect the visibility etc in the future is a really good idea

GeorgeHuntBris commented on Mar 18

This PR will be v useful well done. I think filling numerical columns with mean values and categorical with the most frequent category is a good strategy for imputing values. And as you were saying about trying to get the time to reflect the visibility etc in the future is a really good idea

Similar to your idea about relating time of day to visibility, in the future it could be worth also trying to maybe relate weather condition to the temperature. For example, if it's snowing -> low temperature. But tsh we probs won't have time

GeorgeHuntBris reviewed on Mar 18 View reviewed changes

`rta_backend/rta_api/management/commands/clean_data.py`

GeorgeHuntBris on Mar 18

```
form54data["Time"] = pd.to_datetime(form54data["Time"], format="%I:%M%p", errors="coerce").dt.time
```

Can I just check this converts the time to a date time object %I (Hour - 12 Hour format), %M (Minute), and %p (AM/PM indicator). And then any invalid times entered would just be converted into not a time.

sophiepull on Mar 19 • edited • Author • ...

Almost - the `format="%I:%M%p"` refers to the **current** format of the `form54data["Time"]` column **before** conversion, not how its stored afterwards.

ie in the dataset currently, you can see the times are something like:

```
5:00 PM
3:38 AM
7:12 PM
```

Then after converting it to a datetime object, it just takes the "time" portion of the Pandas `datetime` object (by using `dt.time`). By default Pandas `datetime` is 24 hour format.

Yes - `errors="coerce"` will make anything not in this format a NaT. For example, some of the times in the dataset are `00:00 None` which would be invalid since there is no AM/PM specified.

TLDR: The format in which it is stored after conversion/cleaning is not `%I:%M%p`, but rather it is a 24 hour time.

GeorgeHuntBris on Mar 19

Makes sense, thanks for clarifying

qc23151 previously requested changes on Mar 20 View reviewed changes

qc23151 left a comment

Really great work getting the navbar up and running! I think its best to keep DRY by extracting the navbar into a reusable widgets. Creating a centralized `NavigationBar` component would not only reduce code duplication but also ensure consistency throughout the application.

sophiepull requested changes on Mar 20 View reviewed changes

sophiepull left a comment • edited • ...

I agree with [@qc23151](#) in that the NavBar should really be it's own class. In this current version, it would be very very tedious to update the navbar with new pages since you'd then have to edit the code in every single file!!

I would recommend looking at tutorials online for doing this, since its a very common feature to have. I have found some articles like [this](#) which might be useful.

Further, I think it doesn't really look like a navbar, more like buttons on the top of the page? I think we were thinking of implementing something more like this:

LOGOBAKERY

Services Projects About

Contact

Mohsin-Zeeshan commented on Mar 21 Author • ...

Really great work getting the navbar up and running! I think its best to keep DRY by extracting the navbar into a reusable widgets. Creating a centralized `NavigationBar` component would not only reduce code duplication but also ensure consistency throughout the application.

That is the better approach and have implemented that! Thank you

Example - Sphinx documentation

RailwayTrafficAnalysis

Navigation

Contents:

rta_api package

- Subpackages
- Submodules
- rta_api.admin module
- rta_api.apps module
- rta_api.models module
- rta_api.multinomial_logit module
- rta_api.random_forest_model module
- rta_api.serializers module
- rta_api.tables module
- rta_api.urls module
- rta_api.views module
- Module contents

rta_api.management.commands package

Submodules

rta_api.management.commands.clean_data module

```
class rta_api.management.commands.clean_data.Command(stdout=None,
stderr=None, no_color=False, force_color=False)
```

Bases: **BaseCommand**

handle(*args, **options)

The actual logic of the command. Subclasses must implement this method.

help = "Cleans the Form54 dataset and saves it to a CSV file
'rta_backend/data/Form54_cleaned.csv'."

rta_api.management.commands.clean_data.**clean_dataframe**(df)

Cleans and preprocesses the given DataFrame by performing several data cleaning operations.

Parameters: **df** (*pandas.DataFrame*) – The DataFrame to be cleaned

Returns: The cleaned DataFrame.

Return type: *pandas.DataFrame*

Notes

- The function assumes the presence of globally defined variable *BE_fields* These are the BE_fields we want to consider during cleaning.

rta_api.management.commands.clean_data.**clean_form54**()

Cleans the Form54 dataset.

Parameters: **None**

Returns: This function does not return a value but saves the cleaned

DataFrame to a CSV file 'rta_backend/data/Form54_cleaned.csv'.

Return type: **None**

```
def impute_values(df):
    """
    Fills missing values in columns of a DataFrame using specified strategies.
    Numerical columns are filled with the average (mean) value, found in Form 54.
    Categorical columns are filled with the most frequent category, found in Form 54.

    Parameters
    -----
    df : pandas.DataFrame
        The DataFrame to be cleaned. This CAN be the Form 54 dataset itself - the function will handle this.

    Returns
    -----
    pandas.DataFrame
        A DataFrame with no missing entries in any columns.

    Notes
    ----
    - This function requires access to Form 54 data. Ensure data is in correct directory.
    """
```

rta_api.management.commands.clean_data.**impute_values**(df)

Fills missing values in columns of a DataFrame using specified strategies.

Numerical columns are filled with the average (mean) value, found in Form 54.

Categorical columns are filled with the most frequent category, found in Form 54.

Parameters: **df** (*pandas.DataFrame*) – The DataFrame to be cleaned. This
CAN be the Form 54 dataset itself - the function will handle this.

Returns: A DataFrame with no missing entries in any columns.

Return type: *pandas.DataFrame*

Notes

- This function requires access to Form 54 data. Ensure data is in correct directory.

rta_api.management.commands.clean_data.**read_form54_if_not_read**()

rta_api.management.commands.clean_data.**rename_columns**(df)

Renames columns in the given DataFrame (user input) to the names used in FRA's database. This function should be run on a user input which has variable names in snake case.

Parameters: **df** (*pandas.DataFrame*) – The DataFrame whose columns need
to be renamed.

Returns: The DataFrame with the renamed columns.

Return type: *pandas.DataFrame*

Issues and Solutions - Group Work

Issue: Unequal workload in TB1

- 1 Some team members felt **unsure and uncomfortable** contributing due to a **lack of knowledge** of what was going on in the backend.
- 2 Some team members **not assigning themselves tasks**, and not creating and breaking down their **own issues on Kanban**, leading to uneven workload distribution.

Solutions

- 1 **Project managers** would **clearly outlined what** needed to be done and **how** in each Kanban issue.
- 2 **Project managers** broke down large tasks into smaller ones, assigning the more manageable **smaller tasks** to team members
- 3 Held **in-person peer programming** sessions to receive instant feedback and help from one another

Issues and Solutions - Group Work

Issue: Collaborative coding in GitHub

- 1 (TB1) Initialised the Flutter project multiple times, pushed several initialisations to different branches
- 2 Merge conflicts leading to unintended code deletions
- 3 PR descriptions lacking detail, lack of metadata (Kanban issues, labels, milestones etc)

Solutions

- 1 To prevent future occurrences, had **meeting** to explain project structure & directories, **renaming** of files to **clearer** names
- 2 Conducted **GitHub teaching sessions** on resolving merge conflicts correctly
- 3 Conducted **GitHub teaching sessions** on best practices for making PRs

Issues and Solutions - Technical



#01

Lack of data

Client did not provide data.
Struggled finding data from the UK that would allow us to implement what the client wanted.

Solution

Research into datasets **outside** of UK.
Used dataset from U.S (client was ok with this!)



#02

Train Jargon

Train-specific knowledge and terminology produced issues
-> Frontend input fields refactored often
-> Unsure how to identify outliers in dataset

Solution

Research into train jargon, luckily found large FRA documentation explaining these words.



#03

Creating a model

Time spent researching instead of coding
Deciding between **statistical** model vs **machine learning** model

Solution

Experimenting with both models
Learning about what different models could work

What we would do differently next time

- Using aforementioned solutions to group work issues, **from the start in TB1**
 - Have assigned 2 project managers per sprint, weekly scrum meetings, in-person pair programming
 - Use Teams, PRs, issues, Kanban board
- Ensure **every** team member is clued in with current project **directions and architecture**
 - Done via **thorough code reviews** from the **beginning**, to ensure **everyone knows what every part of the code does**
- **Test driven development**
- These would've allowed more time to do things we haven't done!
 - Spent more time on researching/testing a wider range of possible approaches
 - Add offline capabilities

What we have learned

- **Gained industry-like experience** - how to work in a software engineering team, and working with limited resources from a client.
- **Navigating team dynamics** - learnt how to resolve issues, and collaborate effectively using project management techniques.
- **Independent thinking** - taking initiative and experience in self-learning new programming languages, tools and skills.
- **Value of iterative development** - user testing, clear system requirements, good software engineering practices such as Agile development sprints

Questions

Issues and Solutions - Group work

Issue: Kanban board not being used effectively

- 1 Some team members not assigning themselves tasks
- 2 Not used often by some team members, so unsure what everyone is working on until PR is made
- 3 Assigning tasks for the future but not actually being worked on yet, so others could not pick up those tasks

Solutions

- 1 Introduced **project managers** who create and assign the tasks.
- 2 Use clear task statuses - move tasks to *In Progress* **only when work begins** and **has its own branch**
- 3 Established policy of **not assigning tasks** which are in *To Do*.

INCOMPLETE: Where we could take the project further

Probability of railway incident

Give the probability of a rail incident occurring given specific input variables (not given that an accident occurred)

Offline capabilities

Improve the model

- ✓ Sanitize data!
- ✓ Add more predictor variables
- ⌚ Confidence intervals
- ✓ Improve performance

Experiment with different models

- ⌚ Use different statistical models
- ✓ Experiment with ML

Predictive Model - Old



MNLogit Regression Results							
=====							
Dep. Variable:	y	No. Observations:	100				
Model:	MNLogit	Df Residuals:	-15				
Method:	MLE	Df Model:	110				
Date:	Tue, 12 Nov 2024	Pseudo R-squ.:	0.5265				
Time:	19:49:54	Log-likelihood:	-79.586				
converged:	False	LL-Null:	-168.09				
Covariance Type:	nonrobust	LLR p-value:	5.347e-05				
=====							
	y=accident_type_Explosive-detonation	coef	std err	z	P> z	[0.025	0.975]

temperature		0.3426	0.173	1.978	0.048	0.003	0.682
track_class		-1.2701	0.728	-1.745	0.081	-2.696	0.156
track_density		0.0353	0.026	1.333	0.183	-0.017	0.087
equipment_type		-1.4062	0.745	-1.887	0.059	-2.867	0.054
equipment_attended		-13.1520	10.938	-1.202	0.229	-34.590	8.286
train_speed		0.0203	0.052	0.393	0.694	-0.081	0.122
remote_control_locomotive		-2.4196	1.934	-1.251	0.211	-6.209	1.370
positive_alcohol_tests		0.7272	0.456	1.596	0.110	-0.166	1.620
positive_drug_tests		-0.0542	0.231	-0.234	0.815	-0.508	0.399
visibility_dawn		0.3890	2.408	0.162	0.872	-4.332	5.110
visibility_day		-12.8727	7.824	-1.645	0.100	-28.208	2.462
visibility_dusk		-7.1932	4.673	-1.539	0.124	-16.353	1.966
weather_condition_cloudy		-3.4319	4.284	-0.801	0.423	-11.828	4.964
weather_condition_fog		4.5593	3.617	1.260	0.208	-2.531	11.649
weather_condition_rain		-8.4325	8.459	-0.997	0.319	-25.012	8.147
weather_condition_sleet		4.4000	3.872	1.136	0.256	-3.190	11.990
weather_condition_snow		-12.4009	8.452	-1.467	0.142	-28.966	4.165
track_type_main		-12.7713	7.401	-1.726	0.084	-27.278	1.735
track_type_siding		-7.3775	4.280	-1.724	0.085	-15.765	1.010
track_type_yard		-21.1195	10.990	-1.922	0.055	-42.660	0.421
method_of_operation_Direct Train Control		9.7192	6.285	1.546	0.122	-2.599	22.038
method_of_operation_Signal Indication		-2.9255	3.127	-0.936	0.350	-9.055	3.203
method_of_operation_Yard/Restricted Limits		17.2128	9.505	1.811	0.070	-1.416	35.842

	y=accident_type_Fire/violent rupture	coef	std err	z	P> z	[0.025	0.975]

temperature		0.0207	0.020	1.030	0.303	-0.019	0.060
track_class		0.0461	0.184	0.251	0.802	-0.406	0.514

```
← → ↺ ⓘ http://127.0.0.1:8000/view_model_result/
Pretty-print ☒

{
  "Derailment": {
    "0": 0
  },
  "Fire/violent rupture": {
    "0": 0.176298708
  },
  "Head on collision": {
    "0": 0.0245289262
  },
  "Hwy-rail crossing": {
    "0": 0.1168114646
  },
  "Obstruction": {
    "0": 0.0366203834
  },
  "Other (describe in narrative)": {
    "0": 0.0971297385
  },
  "Other impacts": {
    "0": 0.377528056
  },
  "RR grade crossing": {
    "0": 0.0240681937
  },
  "Raking collision": {
    "0": 0.0369422781
  },
  "Rear end collision": {
    "0": 0.0425723955
  },
  "Side collision": {
    "0": 0.067499856
  }
}
```